

Food Express SQL Evidence Document

Course: COSC457 Database Management Systems

Project: Multi-Vendor Food Delivery Database Application

Database: FoodExpressDB

DBMS: MySQL Workbench

Group Members: Nyle Clements, Nand Chana, Myles Burrows, Franklin Atika, Justice
Moody

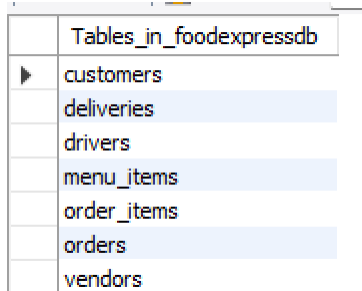
1. DDL: CREATE TABLE

Query:

```
CREATE TABLE customers (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    address VARCHAR(150) NOT NULL  
);
```

Explanation:

This creates the customers table with a primary key, NOT NULL fields, and a UNIQUE email constraint.



Tables_in_foodexpressdb
customers
deliveries
drivers
menu_items
order_items
orders
vendors

2. DML: INSERT

Query:

```
INSERT INTO vendors (vendor_name, category, phone, address, is_active) VALUES  
('Burger Bros', 'Burgers', '410-555-1000', '10 Food St', TRUE),  
('Taco Teleport', 'Mexican', '410-555-2000', '20 Market Ave', TRUE),  
('Pizza People', 'Pizza', '410-555-3000', '30 City Rd', TRUE);
```

Explanation:

This inserts restaurant/vendor records used by the GUI.

Result Grid



Filter Rows:

Edit:







Export/Import:

	vendor_id	vendor_name	category	phone	address	is_active
▶	1	Burger Bros	Burgers	410-555-1000	10 Food St	1
	2	Taco Teleport	Mexican	410-555-2000	20 Market Ave	1
	3	Pizza People	Pizza	410-555-3000	30 City Rd	1
*	NULL	NULL	NULL	NULL	NULL	NULL

3. SELECT

Query:

```
SELECT * FROM customers;
```

Explanation:

This retrieves all customer records.

Result Grid						
		Filter Rows:		Edit:		Export/Import:
	customer_id	first_name	last_name	email	phone	address
▶	1	Nyle	Clements	nyle.clements@gmail.com	410-111-2222	100 Main St
	2	Myles	Burrows	myles.burrows@gmail.com	410-222-3333	200 Oak Ave
	3	Justice	Moody	justice.moody@gmail.com	410-333-4444	300 Pine Rd
	4	Nand	Chana	nand.chana@gmail.com	410-444-5555	400 Hill Rd
	5	Franklin	Atika	franklin.atika@gmail.com	410-555-6666	500 Lake Ave
*	NULL	NULL	NULL	NULL	NULL	NULL

4. WHERE Query #1

Query:

SELECT *

FROM orders

WHERE order_status = 'Completed';

Explanation:

This filters orders by completed status.

Result Grid						
		Filter Rows:		Edit:		Export/Import:
	order_id	customer_id	vendor_id	order_datetime	order_status	total_amount
▶	5	5	2	2026-05-11 10:14:09	Completed	20.98
	6	1	3	2026-05-11 10:14:09	Completed	19.98
*	NULL	NULL	NULL	NULL	NULL	NULL

5. WHERE Query #2

Query:

SELECT *

FROM menu_items

WHERE price > 10;

Explanation:

This filters menu items that cost more than \$10.

Result Grid						
		Filter Rows:		Edit:		Export/Import:
	menu_item_id	vendor_id	item_name	description	price	is_available
▶	4	1	Double Burger	Two beef patties with cheese	12.99	1
	6	2	Chicken Tacos	Three chicken tacos	10.99	1
	7	2	Steak Burrito	Large steak burrito	12.49	1
	9	2	Nacho Bowl	Rice, beans, cheese, and chicken	11.49	1
	10	3	Cheese Pizza	Medium cheese pizza	11.99	1
	11	3	Pepperoni Pizza	Medium pepperoni pizza	13.99	1
	12	3	Veggie Pizza	Vegetable pizza with mushrooms and peppers	14.99	1
*	NULL	NULL	NULL	NULL	NULL	NULL

6. ORDER BY

Query:

```
SELECT *  
FROM menu_items  
ORDER BY price DESC;
```

Explanation:

This sorts menu items from highest to lowest price.

Result Grid						
		Filter Rows:	Edit:		Export/Import:	Wrap Cell Content:
	menu_item_id	vendor_id	item_name	description	price	is_available
▶	12	3	Veggie Pizza	Vegetable pizza with mushrooms and peppers	14.99	1
	11	3	Pepperoni Pizza	Medium pepperoni pizza	13.99	1
	4	1	Double Burger	Two beef patties with cheese	12.99	1
	7	2	Steak Burrito	Large steak burrito	12.49	1
	10	3	Cheese Pizza	Medium cheese pizza	11.99	1
	9	2	Nacho Bowl	Rice, beans, cheese, and chicken	11.49	1
	6	2	Chicken Tacos	Three chicken tacos	10.99	1
	1	1	Classic Burger	Beef burger with lettuce and tomato	9.99	1
	8	2	Quesadilla	Chicken quesadilla with salsa	9.49	1
	2	1	Chicken Sandwich	Grilled chicken sandwich	8.99	1
	5	1	Loaded Fries	Fries with cheese and toppings	5.99	1
	13	3	Garlic Knots	Garlic bread knots with sauce	4.99	1
	3	1	Fries	Seasoned fries	3.49	1
*	NULL	NULL	NULL	NULL	NULL	NULL

7. JOIN Query #1: Customer Orders

Query:

```
SELECT  
    o.order_id,  
    c.first_name,  
    c.last_name,  
    v.vendor_name,  
    o.total_amount,  
    o.order_status  
FROM orders o  
JOIN customers c  
    ON o.customer_id = c.customer_id  
JOIN vendors v  
    ON o.vendor_id = v.vendor_id;
```

Explanation:

This combines customer, vendor, and order information.

8. JOIN Query #2: Deliveries and Drivers

Query:

```

SELECT
    d.delivery_id,
    o.order_id,
    dr.first_name,
    dr.last_name,
    d.delivery_status
FROM deliveries d
JOIN orders o
ON d.order_id = o.order_id
JOIN drivers dr
ON d.driver_id = dr.driver_id;

```

Explanation:

This shows deliveries with their assigned drivers.

Result Grid		Filter Rows:		Export:		Wrap C
	delivery_id	order_id	first_name	last_name	delivery_status	
▶	1	1	Dante	Miller	Assigned	
	4	4	Dante	Miller	Assigned	
	7	7	Dante	Miller	Picked Up	
	9	9	Dante	Miller	Assigned	
	2	2	Alex	Smith	Picked Up	
	5	5	Alex	Smith	Delivered	
	8	8	Alex	Smith	Assigned	
	3	3	Jordan	Lee	Assigned	
	6	6	Jordan	Lee	Delivered	

9. JOIN Query #3: Order Items and Menu Details

Query:

```

SELECT
    oi.order_id,
    mi.item_name,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total
FROM order_items oi
JOIN menu_items mi
ON oi.menu_item_id = mi.menu_item_id;

```

Explanation:

This shows which menu items were ordered, including quantity and line total.

Result Grid					
Filter Rows:					
	order_id	item_name	quantity	unit_price	line_total
▶	1	Classic Burger	1	9.99	9.99
	1	Fries	1	3.49	3.49
	2	Chicken Tacos	1	10.99	10.99
	3	Pepperoni Pizza	1	13.99	13.99
	4	Double Burger	1	12.99	12.99
	4	Loaded Fries	1	5.99	5.99
	5	Quesadilla	1	9.49	9.49
	5	Nacho Bowl	1	11.49	11.49
	6	Veggie Pizza	1	14.99	14.99
	6	Garlic Knots	1	4.99	4.99
	7	Double Burger	1	12.99	12.99
	8	Nacho Bowl	1	11.49	11.49
	9	Pepperoni Pizza	2	13.99	27.98

10. Aggregate Query #1: COUNT

Query:

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Explanation:

This counts the total number of orders.

Result Grid	
	total_orders
▶	9

11. Aggregate Query #2: SUM

Query:

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

Explanation:

This calculates total order revenue.

Result Grid	
	total_revenue
▶	150.86

12. GROUP BY Query #1: Orders Per Vendor

Query:

```
SELECT
```

```

v.vendor_name,
COUNT(o.order_id) AS total_orders
FROM vendors v
LEFT JOIN orders o
ON v.vendor_id = o.vendor_id
GROUP BY v.vendor_name;

```

Explanation:

This groups order counts by restaurant/vendor.

	vendor_name	total_orders
▶	Burger Bros	3
	Taco Teleport	3
	Pizza People	3

13. GROUP BY Query #2: Revenue Per Vendor

Query:

```

SELECT
v.vendor_name,
SUM(o.total_amount) AS revenue
FROM vendors v
JOIN orders o
ON v.vendor_id = o.vendor_id
GROUP BY v.vendor_name;

```

Explanation:

This groups total revenue by restaurant/vendor.

	vendor_name	revenue
▶	Burger Bros	45.45
	Taco Teleport	43.46
	Pizza People	61.95

14. UPDATE: Order Status

Query:

```

UPDATE orders
SET order_status = 'Completed'
WHERE order_id = 1;

```

Explanation:

This updates an order status, matching the restaurant dashboard feature.

Result Grid						
Filter Rows:						
	order_id	customer_id	vendor_id	order_datetime	order_status	total_amount
▶	1	1	1	2026-05-11 10:14:09	Completed	13.48
	2	2	2	2026-05-11 10:14:09	Preparing	10.99
	3	3	3	2026-05-11 10:14:09	Ready	13.99
	4	4	1	2026-05-11 10:14:09	Placed	18.98
	5	5	2	2026-05-11 10:14:09	Completed	20.98
	6	1	3	2026-05-11 10:14:09	Completed	19.98
	7	2	1	2026-05-11 10:14:09	Preparing	12.99
	8	3	2	2026-05-11 10:14:09	Placed	11.49
	9	1	3	2026-05-11 12:03:37	Placed	27.98
*	NULL	NULL	NULL	NULL	NULL	NULL

15. UPDATE: Delivery Status

Query:

UPDATE deliveries

SET delivery_status = 'Delivered',

delivered_at = NOW()

WHERE delivery_id = 1;

Explanation:

This updates delivery status, matching the driver dashboard feature.

Result Grid						
Filter Rows:						
	delivery_id	order_id	driver_id	delivery_status	assigned_at	delivered_at
▶	1	1	1	Delivered	2026-05-11 10:14:09	2026-05-11 14:37:13
	2	2	2	Picked Up	2026-05-11 10:14:09	NULL
	3	3	3	Assigned	2026-05-11 10:14:09	NULL
	4	4	1	Assigned	2026-05-11 10:14:09	NULL
	5	5	2	Delivered	2026-05-11 10:14:09	2026-05-11 10:14:09
	6	6	3	Delivered	2026-05-11 10:14:09	2026-05-11 10:14:09
	7	7	1	Picked Up	2026-05-11 10:14:09	NULL
	8	8	2	Assigned	2026-05-11 10:14:09	NULL
	9	9	1	Assigned	2026-05-11 12:03:37	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL

16. DELETE

Query:

DELETE FROM deliveries

WHERE order_id = 1;

DELETE FROM order_items

WHERE order_id = 1;

DELETE FROM orders

WHERE order_id = 1;

Explanation:

This deletes an order and its related records. It maintains the foreign key integrity.

Result Grid						
		Filter Rows:		Edit:		Export/Import:
	order_id	customer_id	vendor_id	order_datetime	order_status	total_amount
▶	2	2	2	2026-05-11 10:14:09	Preparing	10.99
	3	3	3	2026-05-11 10:14:09	Ready	13.99
	4	4	1	2026-05-11 10:14:09	Placed	18.98
	5	5	2	2026-05-11 10:14:09	Completed	20.98
	6	1	3	2026-05-11 10:14:09	Completed	19.98
	7	2	1	2026-05-11 10:14:09	Preparing	12.99
	8	3	2	2026-05-11 10:14:09	Placed	11.49
	9	1	3	2026-05-11 12:03:37	Placed	27.98
*	NULL	NULL	NULL	NULL	NULL	NULL

SUMMARY

The SQL statements in this project supports the core operations of the Food Express application. The system performs live data retrieval, inserts, updates, deletions, filtering, sorting, joins, and aggregate operations for all stakeholders using the GUI and MySQL database connection.